Phase 9: 端到端训练、文本生成与实验指导

1. TinyStories 训练

1.1 推荐初始配置

```
vocab_size = 10000
context_length = 256
d_model = 512
d_ff = 1344          # ≈ 8/3 * 512 = 1365，取 64 的倍数 = 1344
num_layers = 4
num_heads = 16       # d_k = 512/16 = 32
rope_theta = 10000
total_tokens ≈ 327,680,000
```

需要自己调的超参数：

- learning_rate: 建议从 1e-3, 3e-4, 1e-4 开始 sweep
- warmup_iters: 通常占总步数的 5-10%
- weight_decay: 0.01 ~ 0.1
- $\beta_1 = 0.9$, $\beta_2 = 0.95$ 或 0.999
- $\varepsilon = 1e-8$
- max_grad_norm: 1.0

1.2 计算训练步数

```
total_tokens = 327,680,000
tokens_per_step = batch_size × context_length

# 例: batch_size=32, context_length=256
tokens_per_step = 32 × 256 = 8192
total_steps = 327,680,000 / 8192 = 40,000

# cosine annealing 周期 = total_steps
# warmup ≈ 2000 步
```

1.3 调试策略

第一步：过拟合一个 minibatch

```
# 固定一个 batch，反复训练直到 loss 接近 0
# 如果 loss 不下降，模型有 bug
fixed_x, fixed_y = get_batch(train_data, batch_size=4, ctx_len=256,
device='cpu')
for step in range(1000):
```

```
    logits = model(fixed_x)
    loss = cross_entropy(logits.view(-1, vocab_size), fixed_y.view(-1))
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    if step % 100 == 0:
        print(f"step {step}: loss = {loss.item():.4f}")
# 期望: loss 应该快速下降到接近 0
```

第二步：小数据快速迭代

先在 TinyStories 验证集（22K 文档）上训练一小段时间，确认 loss 在下降。

第三步：全量训练

确认无 bug 后再在完整训练集上运行。

1.4 监控指标

训练过程中应监控：

- 训练 loss（每步或每 N 步）
- 验证 loss（每 K 步评估一次）
- 学习率（确认 schedule 正确）
- 梯度范数（检查是否需要调整 clipping 阈值）
- 生成文本（定性评估）

---

2. 学习率 Sweep

2.1 策略

```
建议的 sweep 值: [1e-2, 5e-3, 3e-3, 1e-3, 5e-4, 3e-4, 1e-4]

对每个 lr:
  - 训练相同的步数
  - 记录 loss 曲线
  - 观察是否发散
```

2.2 观察重点

- **最佳 lr**：通常是"刚好不发散"的最大 lr
- **发散的特征**：loss 突然跳升，NaN，或 loss 不再下降
- **lr 太小的特征**：loss 下降很慢，最终 loss 比最佳 lr 高
- **lr 太大的特征**：初期 loss 下降快，然后突然发散

2.3 Batch Size 实验

```
batch_sizes: [1, 8, 32, 64, 128, ...]

注意:
- 更大的 batch size 通常需要更大的 lr (线性 scaling rule)
- 总 token 数应保持一致 (不同 batch size 对应不同 step 数)
- 或保持 step 数一致但总 token 数不同
```

---

3. 文本生成

3.1 生成流程

```
def generate(model, tokenizer, prompt, max_tokens=256, temperature=1.0,
top_p=0.9):
    model.eval()

    # 编码 prompt
    token_ids = tokenizer.encode(prompt)

    with torch.no_grad():
        for _ in range(max_tokens):
            # 如果序列超过 context_length, 截断
            input_ids = token_ids[-context_length:]
            input_tensor = torch.tensor([input_ids], dtype=torch.long,
device=device)

            # Forward
            logits = model(input_tensor)
            next_token_logits = logits[0, -1, :]  # 最后一个位置的预测

            # Temperature scaling
            next_token_logits = next_token_logits / temperature

            # Softmax
            probs = softmax(next_token_logits, dim=-1)

            # Top-p sampling
            # 1. 排序
            sorted_probs, sorted_indices = torch.sort(probs,
descending=True)
            cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

            # 2. 找到累积概率 > top_p 的位置, 把它们的概率置零
            mask = cumulative_probs - sorted_probs > top_p  # 关键: 减去自身
避免把第一个超过 p 的也去掉
            sorted_probs[mask] = 0.0

            # 3. 重新归一化
            sorted_probs = sorted_probs / sorted_probs.sum()
```

```
            # 4. 采样
            next_token_idx = torch.multinomial(sorted_probs, 1)
            next_token = sorted_indices[next_token_idx].item()

            token_ids.append(next_token)

            # 检查是否生成了 endoftext
            if tokenizer.decode([next_token]) == "<|endoftext|>":
                break

    return tokenizer.decode(token_ids)
```

3.2 Temperature 调整建议

- τ = 0.7 ~ 0.8：较保守，文本更连贯但可能重复
- τ = 1.0：正常 softmax
- τ = 1.2 ~ 1.5：更多样性，但可能不连贯

3.3 Top-p 调整建议

- p = 0.9：常用默认值
- p = 0.95：更多样性
- p = 0.5：非常保守

---

4. 消融实验

4.1 移除 RMSNorm

```
修改 TransformerBlock，跳过 RMSNorm：
  x = x + MHA(x)      # 直接用 x 而不是 RMSNorm(x)
  x = x + FFN(x)

预期:
- 在正常 lr 下可能不稳定或发散
- 需要更小的 lr
- 最终 loss 可能更差
```

4.2 Pre-norm → Post-norm

```
修改为:
  x = RMSNorm(x + MHA(x))   # norm 在 residual 之后
  x = RMSNorm(x + FFN(x))

预期:
- 训练可能不如 pre-norm 稳定
- 可能需要更小的 lr
- 论文 [Xiong et al., 2020] 讨论了原因
```

4.3 移除 RoPE (NoPE)

```
在 MHA 中不对 Q, K 应用旋转。

预期:
- 模型失去位置感知能力
- 但 causal mask 本身提供了一些位置信息（因为 mask 是位置相关的）
- Loss 可能只略微变差
- 生成的文本可能出现语序问题
```

4.4 SwiGLU vs SiLU FFN

```
SiLU FFN（不带 gating）:
  FFN(x) = W2 · SiLU(W1 · x)
  只有 2 个权重矩阵
  为保持参数量可比: d_ff = 4 * d_model

SwiGLU（带 gating）:
  FFN(x) = W2 · (SiLU(W1 · x) ⊙ W3 · x)
  3 个权重矩阵
  d_ff ≈ 8/3 * d_model

预期:
- SwiGLU 通常优于纯 SiLU
- 差距可能不大（取决于数据和训练时长）
```

---

5. OpenWebText 实验

5.1 与 TinyStories 的差异

- OWT 更复杂：多种题材、写作风格、专业术语
- 需要更大的 vocab（32K）
- 同样的模型可能需要更多训练才能获得好的 loss
- Loss 的绝对值通常高于 TinyStories（因为数据更难预测）

5.2 OWT 上的期望

- 相同模型和训练步数下，OWT 的 loss 会比 TinyStories 高
- 这是因为 OWT 的数据更复杂、更多样
- 生成文本质量可能不如 TinyStories（因为模型太小）

---

6. 排行榜优化方向

6.1 常见优化

1. **Weight tying**：让 token_embeddings 和 lm_head 共享权重
    - 减少参数量（可以把省下的参数用在更多层上）
    - 需要注意初始化
2. **更好的初始化**：
    - 用 GPT-3 风格的 scaled init（残差分支的最后一层权重除以 √num_layers）
3. **更大的 batch size**：
    - 利用梯度累积在有限显存下模拟大 batch
4. **torch.compile**：
    - `model = torch.compile(model)` 可以显著加速训练
5. **混合精度训练**：
    - `torch.autocast('cuda', dtype=torch.bfloat16)` 减少显存和计算量
6. **调整模型大小**：
    - 在固定训练时间内，更宽/更浅 vs 更窄/更深哪个更好?
    - 参考 scaling laws 论文

6.2 实验方法论

1. 先在 TinyStories 或 OWT 子集上验证修改
2. 跟踪验证 loss 而不是训练 loss
3. 用 wandb 记录所有实验
4. 保存最佳 checkpoint

---

7. 常用命令参考

```
# 运行所有测试
uv run pytest tests/ -v

# 运行特定测试
uv run pytest -k test_linear -v
uv run pytest -k test_transformer_lm -v
uv run pytest -k test_train_bpe -v
uv run pytest -k test_tokenizer -v

# 运行测试并显示输出
uv run pytest -k test_xxx -v -s

# 性能分析
uv run python -m cProfile -s cumtime your_script.py

# 使用 scalene
uv run scalene your_script.py
```